

```
#pragma once

#include "DoublyLinkedList.h"
#include "DoublyLinkedListIterator.h"

#include <stdexcept>

template<typename T>
class List
{
private:
    // auxiliary definition to simplify node usage
    using Node = DoublyLinkedList<T>;

    Node* fRoot;    // the first element in the list
    size_t fCount;  // number of elements in the list

public:
    // auxiliary definition to simplify iterator usage
    using Iterator = DoublyLinkedListIterator<T>;

    ~List() // ➤
    {
        destructor - frees all nodes
        while (fRoot != nullptr)
        {
            if (fRoot != &fRoot->getPrevious()) // more ➤
            {
                Node* lTemp = const_cast<Node*>(&fRoot->getPrevious()); // ➤
                // select last

                lTemp->isolate(); // ➤
                // remove from list
                delete lTemp; // free
            }
            else
            {
                delete fRoot; // ➤
                // free last
                break; // ➤
                // stop loop
            }
        }
    }

    void remove(const T& aElement) // remove ➤
```

```
first match from list
{
    Node* lNode = fRoot;                                // Are there still nodes available?
    start at first

while (lNode != nullptr)                                // Have we found the node?
{
    if (**lNode == aElement)                            // stop the search
    {
        break;
    }

    if (lNode != &fRoot->getPrevious())                  // not reached last
    {
        lNode = const_cast<Node*>(&lNode->getNext());    // go to next
    }
    else
    {
        lNode = nullptr;                                // stop search
    }
}

// At this point we have either reached the end or found the node.
if (lNode != nullptr)                                   // We have found the node.
{
    if (fCount != 1)                                     // not the last element
    {
        if (lNode == fRoot)
        {
            fRoot = const_cast<Node*>(&fRoot->getNext()); // make next root
        }
    }
    else
    {
        fRoot = nullptr;                                 // list becomes empty
    }

    lNode->isolate();                                    // isolate node
}
```

```

        delete lNode;                                //
        release node's memory
        fCount--;                                    //
        decrement count
    }
}

////////////////////////////////////
/// PS3
////////////////////////////////////

// P1 - Problem 1 Implementation

List() : fRoot(nullptr), fCount(0)                  //
    default constructor
{
    // Initialize empty list
}

bool empty() const                                  // Is
    list empty?
{
    return fCount == 0;
}

size_t size() const                                 //
    list size
{
    return fCount;
}

void push_front(const T& aElement)                   // adds
    aElement at front
{
    Node* lNewNode = new Node(aElement);            //
        create new node on heap

    if (fRoot == nullptr)                           //
        empty list
    {
        fRoot = lNewNode;                            // new
        node becomes root
    }
    else
    {
        fRoot->push_front(*lNewNode);                //
            insert before root
        fRoot = lNewNode;                            // new
        node becomes new root
    }
}

```

```
}

    fCount++;
    increment count
}

Iterator begin() const
    return a forward iterator
{
    return Iterator(fRoot);
}

Iterator end() const
    return a forward end iterator
{
    return Iterator(fRoot).end();
}

Iterator rbegin() const
    return a backwards iterator
{
    return Iterator(fRoot).rbegin();
}

Iterator rend() const
    return a backwards end iterator
{
    return Iterator(fRoot).rend();
}

// P2 - Problem 2 Implementation

void push_back(const T& aElement)
    aElement at back
{
    Node* lNewNode = new Node(aElement);
    create new node on heap

    if (fRoot == nullptr)
        empty list
    {
        fRoot = lNewNode;
        node becomes root
    }
    else
    {
        // Insert at the end (before fRoot in circular structure)
        const_cast<Node*>(&fRoot->getPrevious())->push_back(*
            lNewNode); // insert after last node
    }
}
```

```
}

    fCount++;
    increment count
}

// P3 - Problem 3 Implementation

const T& operator[](size_t aIndex) const           // list
indexer
{
    if (aIndex >= fCount)                          //
        check bounds
    {
        throw std::out_of_range("Index out of bounds.");
    }

    const Node* lCurrent = fRoot;                  //
    start at root

    for (size_t i = 0; i < aIndex; i++)            //
        traverse to index
    {
        lCurrent = &lCurrent->getNext();           // move
        to next node

    }

    return **lCurrent;                             //
    return element at index
}

// P4 - Problem 4 Implementation

List(const List& aOtherList) : fRoot(nullptr), fCount(0) // copy
constructor
{
    *this = aOtherList;                            // use
    assignment operator
}

List& operator=(const List& aOtherList)            //
assignment operator
{
    if (this != &aOtherList)                       //
        avoid self-assignment
    {
        this->~List();                             //
        destroy current content
        fRoot = nullptr;                          //
    }
}
```

```
        reset root
        fCount = 0;
        reset count

        // Copy elements from other list
        for (const T& element : aOtherList)
        {
            push_back(element);
        }

        return *this;
    }

// P5 - Problem 5 Implementation

List(List&& aOtherList) : fRoot(aOtherList.fRoot), fCount
(aOtherList.fCount) // move constructor
{
    aOtherList.fRoot = nullptr;
    steal resources
    aOtherList.fCount = 0;
    leave other empty
}

List& operator=(List&& aOtherList) // move
assignment operator
{
    if (this != &aOtherList) //
    avoid self-assignment
    {
        this->~List(); //
        destroy current content

        fRoot = aOtherList.fRoot; //
        steal resources
        fCount = aOtherList.fCount;

        aOtherList.fRoot = nullptr; //
        leave other empty
        aOtherList.fCount = 0;
    }

    return *this;
}

void push_front(T&& aElement) // move
push_front
{
```

```

    Node* lNewNode = new Node(std::move(aElement));           //      ↗
        create node with moved element

    if (fRoot == nullptr)                                     //      ↗
        empty list
    {
        fRoot = lNewNode;                                     // new  ↗
        node becomes root
    }
    else
    {
        fRoot->push_front(*lNewNode);                         //      ↗
            insert before root
        fRoot = lNewNode;                                     // new  ↗
        node becomes new root
    }

    fCount++;                                                 //      ↗
        increment count
}

void push_back(T&& aElement)                                  // move  ↗
{
    create node with moved element
    Node* lNewNode = new Node(std::move(aElement));           //      ↗

    if (fRoot == nullptr)                                     //      ↗
        empty list
    {
        fRoot = lNewNode;                                     // new  ↗
        node becomes root
    }
    else
    {
        // Insert at the end (before fRoot in circular structure)
        const_cast<Node*>(&fRoot->getPrevious())->push_back(*lNewNode); //      ↗
        // insert after last node
    }

    fCount++;                                                 //      ↗
        increment count
}
};

```